

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS700 Research Report

A Per-User Behavioural Baseline Framework for Insider Threat Detection in Resource-Constrained Environments

Student number: u20748052

Supervisor(s):
Prof. Jan Eloff

Author: Selepe Sello

Live dashboard: `https://cos700research.tebogoyungmercykay.co.za`
Source code: `https://github.com/TebogoYungMercykay/insider_threat_cos700`

November 2026

Abstract

Many lightweight detectors compare every employee with the whole organisation, using one shared picture of normal called a population baseline, and this fails in two ways at once. Some users are very busy for good reasons, such as IT administrators, and their days fill the queue of alerts that a security analyst must check. Meanwhile a low-and-slow insider, who changes their behaviour slowly and stays within population norms, is never found. This report designs, builds and tests a different lightweight framework, which learns a profile of each user from that user's own history and uses a role-group profile instead if the history is too short. One anomaly model then scores each day by how far it strays from the profile, so a day is judged unusual for that person, not unusual in general.

The framework was tested on the full CERT Insider Threat Test Dataset r6.2, which has 4 000 users and 135 million log events. Every detector was allowed the same small number of alerts, 403, which is 0.1% of all user-days, where a user-day is one user on one day. In the test period, the later months kept out of training, the framework found 21 of the 38 malicious days and both insiders. The comparison detector, a Random Forest classifier, found 8 days and one insider. The framework spent 3.5% of its alerts on IT administrators while the Random Forest spent 22.6%, and it trained in eight seconds against the Random Forest's five minutes. Comparing days with the whole workforce instead of the user's own history found only 3 days. The comparison was then repeated ten times on the same data with different random starting choices, called seeds, and in every run the framework found 20 or 21 of the 38 days and beat every population detector on recall, the share of malicious days found, and on administrator alert share. A personal baseline is stable after about 30 days of history and useful after about 60, so the framework's rule of trusting a profile after 14 days sets the bar too low, and the report recommends raising it to about 60 days. The framework explains each alert by naming a feature, and on 95% of malicious days the top reason was a feature the insider actually used. The Random Forest's SHAP explanations did this on at most 50%. Two limits are discussed with the results, that the test window holds only two insiders, and that the data is synthetic, made by a program, not collected from real people.

Keywords:

insider threat detection, anomaly detection, per-user baseline, CERT r6.2.

1 Introduction

Insider threats are among the slowest security incidents to detect and the costliest to suffer. In IBM's 2024 study of 604 breached organisations, attacks by malicious insiders took an average of 287 days to identify and contain. They were also the costliest way for an attack to begin, at an average of USD 4.99 million [1]. The danger is built in. An insider already has a valid login and valid access and knows how the organisation works, so the defences that keep outsiders out of the network do not apply, the misuse looks like ordinary work, and it can run unseen for months while the damage grows. Detection therefore means telling malicious use of that access from routine use [2, 3].

Lightweight detectors answer this with one shared picture of normal, and it fails in two directions. The picture is learned from the whole workforce and is called a baseline, and whoever departs from it is flagged. The published classifiers for the CERT data compared in this report work this way [4]. The CERT data is a public synthetic dataset of employee logs, named after the CERT Division of Carnegie Mellon University, which released it. First, some employees have naturally intense work, such as IT administrators, and they are flagged far more often than their share of the workforce (Section 6.3). Just by doing their jobs they fill the alert queue, the list of flagged days that security analysts check, and analysts learn to discount false alarms they see every day [5]. Second, an insider may change their behaviour gradually, so that each day stays within what the population calls ordinary, and that insider is never found, even when the change is large against their own history. This is the "low-and-slow" pattern of data theft in the CERT guide to insider threats [2].

Per-user approaches exist, but they need costly equipment and software. Commercial User and Entity Behaviour Analytics (UEBA) platforms profile each user and their peer group, the colleagues in the same role [6], and academic work has also moved to per-user models of the CERT data [7, 8]. Both routes need equipment an organisation must buy, run and staff, and this report calls an organisation that cannot do so resource-constrained. One route needs a SIEM feeding a licensed analytics platform, where SIEM stands for security information and event management and a SIEM collects an organisation's logs in one place. The other route needs a large neural network trained on a live stream of logs, where a neural network is a model of many connected simple units that learns from large amounts of data. None of the works reviewed in Section 5 builds per-user baselines from exported log files on ordinary hardware, or tests them against the population-level practice they would replace.

This report designs, builds and tests such a framework, where each user's

normal behaviour is summarised from their own history and their role group. Scoring has two steps. First, each kind of activity on a day is turned into a deviation from the user's own usual value, and the thirteen deviations of a day form its deviation vector. Second, a single anomaly model gives the day one score from the whole vector, so the unit of detection is the user-day. An anomaly model learns what normal data looks like without labels, which are markers saying which days were malicious. The largest deviations are shown to the analyst as the reasons. The framework is tested on the full CERT Insider Threat Test Dataset r6.2 [9, 10], with 4 000 users and 135 million log events, and it is compared against the Random Forest and against control detectors. The Random Forest is a vote of many decision trees, where each tree is a chain of yes or no questions on the features. It is a population-level classifier, a model that learns from labelled examples, and it was chosen as the comparison because Noever's earlier study found it the strongest for this data [4]. The control detectors change only one thing at a time, so any difference can be traced to its cause.

Contributions. This report makes the following contributions:

1. A lightweight per-user behavioural baseline framework made of a Profile Builder, a Role-Aware Normaliser and an Anomaly Scorer, which trains on the full 4 000-user dataset in about eight seconds on a laptop.
2. A data pipeline that reads the 93 GB of raw r6.2 logs once and reduces them to an 8.6 MB table with one row per user per day. The table is frozen in one 8.7 MB file with the roles, the split of days into training and test, and exact day-level labels from the answer key, the dataset's official list of malicious events.
3. A controlled comparison at a fixed alert budget on one date-ordered split, which covers 16 detectors on the full population and 45 detector settings on a 400-user sample. These are 36 framework variants and 9 comparison detectors, and they include the same algorithm with a population reference, a no-model detector that ranks by activity volume, and four of the classifier families compared by Noever [4].
4. Three follow-up studies, which check consistency across ten repeated runs with different random choices, called seeds, measure the history a personal baseline needs, SQ3, and ask whether each system's explanations name what the insider did, SQ5.
5. A record of the negative and unexpected results, and a review of what was built against the research proposal, the plan submitted before this work began.

2 Problem Statement

The published lightweight detectors for this data [4, 11] judge every user against one baseline for the whole workforce, and two failures follow. A high-activity user such as an IT administrator is flagged, because their activity is unusual for the population but normal for their role. An insider who spreads data theft over many ordinary days is not flagged, because no single day exceeds a population threshold, yet their behaviour has changed sharply [2].

Both failures have one cause. The detector has no picture of what is normal *for this individual in this role*. No published, lightweight framework fills the gap. Such a framework would build individual baselines from standard log exports, fall back to the role group when a person's history is too short, and need no costly equipment. The study covers behavioural features from logs on one public dataset, CERT r6.2.

2.1 Research Questions

The hypothesis is that a user's own baseline does better than a population baseline in two ways, when each user-day is scored against that baseline, with role-group fallback. It finds more low-and-slow insider activity, it spends fewer alerts on high-privilege users, and it does so at a cost a laptop can carry. Five sub-questions test it, answered in Sections 6.2, 6.3, 6.6, 6.7 and 6.8. In SQ2, role-aware normalisation means the whole framework.

SQ1 To what extent does a per-user behavioural baseline reduce missed detections of low-and-slow insider behaviour compared to a population-level baseline?

SQ2 To what extent does role-aware normalisation reduce the false positive rate for high-privilege legitimate users compared to population-level evaluation? Role-aware normalisation rescales numbers against the group's usual values, and the false positive rate is the share of innocent days wrongly flagged.

SQ3 What is the minimum amount of historical log data needed to construct a stable and reliable individual baseline?

SQ4 How does the computational cost of per-user baseline construction compare to population-level model training, and is it feasible in a resource-constrained deployment?

SQ5 What additional context does a per-user deviation score give a security analyst relative to a population-level score, and does it reduce investigation time?

3 Methodology

This research accepts a claim only when a measurement supports it, and the measurements come from a quantitative, controlled experiment. Quantitative means that every conclusion rests on measured numbers, and controlled means that only one thing differs when two systems are compared. A per-user framework and a set of population-level detectors are trained on the same data and scored on the same later period, which none of them saw in training, and they are compared on measures fixed in advance, at the same alert budget. The software was built in repeated rounds, and automated tests checked each round. A comparison on rare events can report a misleading number in specific ways, so the tests guard against those ways. The detectors and measures named in this section are defined in Section 4.

Figure 1 summarises the work as five steps. Step 1 reduces the raw logs to one row per user and day, and Step 2 attaches roles and labels and splits the rows by date (Section 3.1). Step 3 fits the per-user framework (Section 3.2), while Step 4 fits the population detectors it is compared with (Section 3.3). Step 5 scores every detector at the same alert budget (Section 3.4). Step 5 also runs the follow-up studies (Section 3.5).

3.1 Data Preparation

Step 1 Reading the logs. The logs are too big for memory, so they are read in pieces. The extracted r6.2 logs total 93.2 GB, and the web log alone is 84 GB, so they cannot be loaded into 32 GB of memory. Sizes are in binary units throughout this report, so 1 GB is 1024 times 1024 times 1024 bytes. A feature is one number that describes one aspect of a user's day, and an example is the number of emails sent. Every feature the framework uses is a count, a sum or a count of distinct values, each per user per day. Such features can be computed in a single pass through each file, and only one row per user-day is held in memory, not every event. The log reader therefore cuts each file into pieces, and each piece starts and ends on whole lines. The pieces are read in parallel, the calendar day and hour are taken directly from the fixed-width time stamps, and the piece totals are then added together. One feature cannot be added across pieces, the number of distinct web domains, so it is carried as a compact set of domain identifiers per user-day, and the sets are combined when the pieces are merged.

The number of parallel workers changes only the speed, where a worker is one copy of the program running alongside the others. Every experiment ran on one laptop with 14 cores and 32 GB of memory. On it, 12 workers reduce the 135 million events to 1 394 010 user-days in 3.3 minutes, while

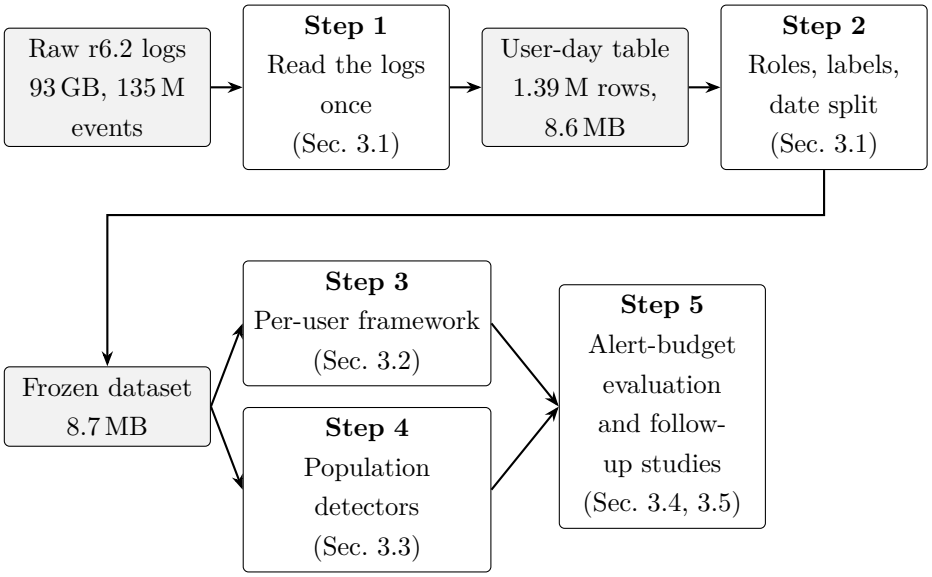


Figure 1: The pipeline in five steps, where each step is described in the section named in its box. In Step 1 the raw logs are read once and reduced to one row per user and day. In Step 2 that table is frozen into a single 8.7 MB file with roles, labels and the date split, where frozen means written once, fingerprinted and never changed. Steps 3 and 4 train the per-user framework and the population detectors it is compared with, and they use that file alone. Step 5 scores every detector under the same evaluation.

with 4 workers the same pass took 15 minutes and used at most 2.8 GB of memory. These two timings were noted in the pipeline's documentation at the time, and they are not from a saved run record. The table behind every result in this report was built with 8 workers, and the worker count changes only the speed, never the content, which the tests confirm. The content of the table does not depend on the number of workers, and tests require the result from pieces to equal a single-pass result exactly. Two builds of the table on different days carry the same fingerprint. This fingerprint is a hash, which is a short code computed from the file, and it changes if any byte of the file changes.

The features. Each user-day carries thirteen behavioural features (Table 1). An early version had coding mistakes in four of them, and those four always gave zero or the wrong value on the real data. The mistakes were corrected before any experiment was run, and tests now lock the corrections

in (Table 14). One feature needs a definition. An email is external if any recipient has an outside address, where recipients are read from the to, cc or bcc fields. The domain of an address is the part after the @, and an address is outside when its domain differs from the organisation's own, which is read from the LDAP employee directory.

Table 1: The thirteen per-user-day behavioural features, with the size of the log file each one is read from.

Log	Size	Feature	Definition
logon	230 MB	`n_logon`, `n_logoff`	Logon and logoff events
		`n_afterhours_logon`	Logons before 07:00 or from 19:00
device	133 MB	`n_device_connect`	Removable-device connections
file	1.24 GB	`n_file_events`	File operations
		`n_file_to_removable`	File operations whose target is removable media
http	84.0 GB	`n_http_events`	Web requests
		`n_http_upload`	Web uploads
		`n_distinct_domains`	Distinct web domains (sites) visited
email	7.53 GB	`n_email_sent`	Emails sent (the log also records emails viewed, which are not counted)
		`n_email_external`	Sent emails with at least one external recipient
		`n_email_attachments`	Attachments on sent emails
		`total_email_size`	Total bytes of sent email

Step 2 Labels. The labels come from the official answer key. A label marks a user-day as malicious or benign, where benign means not malicious. Each malicious log line is mapped to its user and calendar day, which gives exact day-level labels. There are 44 malicious user-days belonging to five insiders. Three earlier labelling mistakes are recorded in Table 14. The first read the answer keys of every dataset release instead of r6.2 only, and the second counted the 24 recipients of one email, sent by the scenario 3 insider of Table 4, as attackers. The third marked every day between an insider's first and last event.

Step 2 Roles and the date split. Each user-day is joined to its user's role, department and supervisor, which come from the final LDAP snapshot, dated May 2011. The data is then split by date, and the split falls at 70% of the 516 calendar days, so the first 361 days train and the rest test. The training part is 990 765 user-days up to 28 December 2010, and every detector learns from it. The test part is 403 245 user-days from 29 December 2010, and every detector is scored on it and never sees it in training. A random split would be wrong here, because it would let a detector learn from days after the ones it is scored on.

The split has a result that shapes everything that follows. Insiders are sorted by the span of their malicious activity, where a span longer than three weeks is low-and-slow and a shorter span is a burst. Span is the working rule used for the low-and-slow pattern, and it gives three burst insiders and two low-and-slow ones. All three burst insiders act before the split, while both low-and-slow insiders act after it (Table 4). So the test window contains 38 malicious days, all low-and-slow, and the training window contains six, all burst. The supervised detectors learn from labelled examples, so they must learn what malicious looks like from six examples, and those six are a different kind of attack. Moving the split could put a burst insider in the test window, but then there would be no malicious training rows at all, so the limit is reported rather than engineered away. The split also means the test window measures exactly the behaviour SQ1 asks about.

Every day count in this report is in active days, and an active day is a day with at least one logged event for that user. Every one of the 4 000 users has at least 21 active days in the training window, and the average is 248. Of these users, 3 744 appear in the test window, while the other 256 have no logged activity after the split date.

3.2 The Per-User Framework

The framework has three parts, and Figure 2 gives the whole procedure.

Profile Builder. The Profile Builder records what is normal for each user. For every user and every feature, it computes the mean, the standard deviation and the 25th, 50th, 75th, 90th and 99th percentiles, all from that user's training days. The standard deviation is the usual spread of the values, and the 90th percentile is the value that 90 percent of the user's days fall below. The percentiles are stored for the explanation, which can then say what a usual day looks like for this person, the median, and what a busy day looks like, the 90th percentile.

Role-Aware Normaliser. The normaliser decides whose normal a day is compared with. The same statistics are computed for every role group and for the whole population. The cold-start threshold is 14 active days of history, and a user with less history is measured against their role group, while a role absent from the training window falls back to the population. Every r6.2 user has a role in the directory, so no user lacks one, and every user has at least 21 training days, so the fallback is never triggered in the main experiments. The history study of Section 6.6 uses it on purpose. The reference frame is whose normal a day is compared with, and it is a single setting named *scope*, whose values are `population`, `role` or `personal`. A second setting is the *statistic*, which chooses the z-score or the median form of the deviation. These two words appear in the name of every framework detector in the result tables of Section 6, for example `peruser-ocsvm-personal-zscore`. The scope setting means the three frames share one piece of code and differ in nothing else, which makes the comparison of the framework's parts a clean test of the reference frame.

Anomaly Scorer. Scoring has two steps. First, each feature of a user-day is turned into a deviation, which is the z-score z_f or the median form z_f^{median}, measured against the row's reference profile. The thirteen deviations form the day's deviation vector. Second, the anomaly model scores that whole vector, so each user-day receives one score, and the per-feature deviations are kept for the explanation. A feature the user has never shown has a spread of zero, and that spread is raised to one, so the first-ever write to removable media counts as a deviation of one per file and is not a division by zero. One anomaly model is trained on the training deviation vectors, where training is also called fitting, and the model then scores the test vectors. Those vectors already express "how unusual for this person", so a single shared model behaves as a per-user detector and no per-user model has to be trained or stored. LOF and the One-Class SVM could not be fitted on all 990 765 rows in reasonable time on the laptop, so every anomaly model is fitted on the same random sample of 50 000 training rows, and this includes the Isolation Forest. The sample is drawn with a fixed seed, and the population anomaly controls use the same sample size. Profiles are always built on the full training window, and every test row is scored.

The ranking always uses the model's own score. The dashboard is the browser application of Section 6.9, and it shows a 0 to 1 score, which is for display only and keeps the same order. An early version capped scores at 1.0, which tied the whole top of the ranking (Table 14).

Input: training user-days D_{train}, test user-days D_{test}, scope, statistic, anomaly model M, cold-start threshold $h_{\min}$

Output: one anomaly score per test user-day, and its explanation

```
# Profile Builder + Role-Aware Normaliser
# (mean, std and p25..p99 per group)
P_user = stats_by(D_train, "user")
P_role = stats_by(D_train, "role")
P_pop  = stats(D_train)

def reference(row):   # scope = personal
    # (role scope: P_role, population: P_pop)
    # history = active training days
    if history(row.user) >= h_min:
        return P_user[row.user]
    if row.role in P_role:
        return P_role[row.role]
    return P_pop

def deviations(rows):
    c, s = centre_scale(reference, rows)
    s[s <= 0] = 1.0   # never-seen feature
    return (rows.X - c) / s

# Anomaly Scorer: one model on deviations
M.fit(subsample(deviations(D_train), 50000))
Z = deviations(D_test)
# library scores higher = more normal, so flip
score   = -M.decision_function(Z)
explain = top_k(abs(Z), k=3)
```

Figure 2: The per-user framework as one procedure, fit and score. Profiles are built per user, role and population, each row becomes a deviation vector against its reference profile, and one anomaly model scores the vectors.

3.3 Comparison Detectors

Every detector works the same way from the outside. It is fitted on the training part and returns one score per test row, where a higher score means more suspicious, and no detector sets its own cut-off. Table 2 lists the families.

Table 2: The detector families in the comparison.

Family	**Detectors**	**Purpose**
Floors	`random`, `volume`	No-model detectors. `volume` scores a day by the sum of its thirteen feature values and `random` gives every day a random score. A detector that cannot beat them is not worth its extra effort
Population, supervised	`rf`, `dt`, `gb`, `lr`	Random Forest, decision tree, gradient boosting and logistic regression, four of the classifier families Noever compares [4]. `rf` is the research proposal's comparison detector. Trained on a SMOTE-rebalanced set (Section 4), raw features
Population, unsupervised (no labels)	`pop-iforest`, `pop-lof`, `pop-ocsvm`	Isolation Forest, LOF and One-Class SVM with a population reference, the control that isolates the reference frame
Per-user framework	`peruser-<algorithm>-<scope>-<statistic>`	The framework across 3 algorithms, 3 scopes, 2 statistics, with and without a size channel (defined below)

The supervised comparison detectors are four classifiers, which are a Random Forest of 300 trees, a decision tree, a histogram-based gradient boosting classifier and a logistic regression. The logistic regression works on features rescaled to a common range. A decision tree is a single tree of threshold rules, while gradient boosting is a group of shallow trees, where each tree corrects the errors of the previous ones. The faster histogram form was used,

because the standard form could not be trained on the two million SMOTE rows in reasonable time. Logistic regression adds up weighted features to give a probability, and all four are SMOTE-balanced.

Detector names in the tables and figures follow one pattern. `rf`, `dt`, `gb` and `lr` are the Random Forest, the decision tree, gradient boosting and logistic regression, and `pop-iforest`, `pop-lof` and `pop-ocsvm` are the Isolation Forest, LOF and One-Class SVM with a population reference. A framework detector is named `peruser-<algorithm>-<scope>-<statistic>`, so `peruser-ocsvm-personal-zscore` is the One-Class SVM on personal z-scores and `peruser-iforest-role-zscore` is the Isolation Forest on role-group z-scores. A trailing `-mag` marks the size channel, defined after Table 2. `volume` ranks days by total activity, while `random` ranks them at random. In short labels, OCSVM stands for One-Class SVM and iForest for Isolation Forest.

The population unsupervised detectors are what make the main comparison mean something, and they are trained without labels. `pop-ocsvm` and the framework's `peruser-ocsvm-personal-zscore` run the same One-Class SVM with the same parameters on the same rows, and they differ in one thing only. One expresses each row relative to the population, while the other expresses it relative to the user. Without that control, a framework that beat the Random Forest would have changed two things at once, the algorithm and the reference frame.

The *size channel* is an optional variant that adds the raw features to the deviation vector, and those raw features are rescaled against the population. The anomaly model can then require two things of a day, which must be unusual for the person and large in absolute terms.

3.4 Evaluation Method

Alert budget. An alert budget is the share of test user-days a detector may flag. Every detector is evaluated at a budget of 0.1%. That is 403 alerts over five months across 4 000 users, and it is about three alerts a day. Budgets from 0.01% to 5% are also tried, and they show how recall rises as the analyst accepts more alerts.

Ties. Ties are the normal case on this data, and a tie is when many rows share one score. The Random Forest is trained on six malicious rows, and it gives 402 941 of the 403 245 test rows a probability of exactly zero. The first version flagged every row tied at the cut, and on the 400-user sample this let the Random Forest flag all 40 360 of its test rows, while the 1% budget

there was 404. Ties are now broken by a random shuffle fixed by the seed, so exactly the budgeted number of rows is flagged, and the choice among equals does not depend on the detector or on the row order. The share of a detector's budget decided this way is its *tie fraction*. A first version of that measure counted one row too many, and it was corrected.

Metrics. The research proposal's measures are reported at the budget, and they are recall, precision, F1, the false positive rate and ROC-AUC, where the false positive rate is given overall and among high-privilege users. Average precision is the main ranking measure, and it ranks detectors by the very top of their queue. Recall and insider-level recall at the budget say how much malicious activity the budget catches, and the main claims rest on those. SQ2 is answered mainly by the alert share below, which is what fills the analyst's queue, and the false positive rate is given as well. Four more measures were forced by the data:

- **Insider-level recall**, whether each insider was found at all. Day-level recall over 38 malicious days moves in steps of 2.6 points, because one day is 1 in 38. This evaluation assumes that an analyst acts on whether the person was found.
- **High-privilege alert share**, the share of the alert queue spent on high-privilege users. These users are 2.43% of the test rows, so a small subgroup false positive rate can still mean they fill most of the queue. The alert share is the measure of alert fatigue used here, and the interview study of security analysts found false alarms to be their main complaint [5].
- **Alerts ahead**, the number of user-days ranked above an insider's best-ranked malicious day. It needs no cut-off, and it is how far down the queue an analyst must read to reach that insider, so it stands in for investigation effort in SQ5.
- **Detection delay**, the days from an insider's first malicious day to the first malicious day flagged at the budget. The problem statement is about how long insiders go undetected, and this measures it directly.

High-privilege users are identified from the LDAP role, where the code matches administrator role names. In r6.2 this subgroup is exactly the ITAdmin role, which is 100 users and 2.43% of test user-days. The research proposal named database administrators, system operators and security analysts, but none of these roles exists in r6.2. Every measure is computed on the full test part, never a sample.

3.5 Follow-up Studies

Seed consistency. The seed study re-runs seven detectors under ten seeds. A seed is the number that fixes every random choice a run makes, so the same seed gives the same run again, and the main comparison uses one seed. The seven detectors are the framework, its role-scope and Isolation Forest variants, the two population anomaly controls `pop-ocsvm` and `pop-iforest`, the Random Forest and the volume floor. Each seed re-draws the training sample, the forests, SMOTE and the tie-break, while the split and features stay fixed. The framework is compared with each other detector on three measures, which are average precision, recall and high-privilege alert share. The comparisons use paired statistical tests at the 5 percent level, and they are corrected with the Holm correction across all 18 comparisons, which is 6 other detectors times 3 measures.

History length. This study asks how much history a personal baseline needs. Each user's training data is cut to their most recent N active days, where the values are $N \in \{3, 7, 14, 30, 60, 90, 120, 180\}$ and the full window, which is 248 active days on average. This imitates an organisation that has collected logs for N days when monitoring starts. The framework is refitted on the shortened data and scored on the unchanged test window. The cold-start threshold is lowered to N where necessary, so every user is really measured against an N-day personal profile and no user silently falls back to their role. Two properties are measured. *Stability* has two parts. The first is how far the N-day per-user means sit from the full-history means, and this distance is in full-history standard deviations. The second is the share of the test window's *benign* user-days with any feature beyond $|z| = 3$ under the N-day profile, where a value of $|z| = 3$ is three spreads from the usual value. This second part is the load of false deviations an analyst would be left with. *Usefulness* is detection at the 0.1% budget, and it is compared with the role baseline, which needs no personal history at all. Both the One-Class SVM and Isolation Forest scorers are run under three seeds.

Explanation accuracy. The answer key gives an exact record of what each insider did, and every answer-key row is an exact copy of a malicious log line. Those rows are replayed through the same code that reads the logs, which gives, for each malicious user-day, exactly which features the malicious events changed. This is the ground truth for "what the insider did that day", where ground truth means the official record, and no hand-made mapping is needed. Each of the 38 malicious test days is explained five ways, and each explanation is the top three features an analyst would be shown. Every

explanation is scored against this ground truth:

- the framework's, the largest $|z|$ against the user's own history.
- the same against the role group.
- the same against the population, which is the control that isolates the reference frame from the explanation method.
- per-row SHAP values, defined in Section 4, of the research proposal's Random Forest, ranked by size ignoring sign.
- the same SHAP values restricted to features that push towards an alert, where a negative value is a reason against one.

The measures are hit@1, hit@3 and precision@3. Hit@1 is whether the top-named feature is one the insider used, hit@3 is whether any of the top three is, and precision@3 is the share of the top three that are. They are compared with the exact expected rate of a random explanation, which names m of the 13 features. Its expected rate is $1 - \binom{13-g}{m} / \binom{13}{m}$, averaged over the days, where g is the number of features the insider's events actually changed that day and m is 1 for hit@1 and 3 for hit@3.

3.6 The Code and Its Settings

The system is written in Python 3.14, and it uses pandas 3.0, NumPy 2.5, scikit-learn 1.9 [12], imbalanced-learn 0.14 (SMOTE), SHAP 0.52 and SciPy 1.18. It has a dashboard built with Streamlit, which is a Python framework for browser applications. The source code is public at the address on the title page, and its documentation lists the command behind every table in this report. Table 3 lists the defaults. All experiments ran on a consumer laptop under Linux, with 14 cores, 32 GB of memory and no graphics processor.

Every table in this report traces back to the run that produced it, because every run writes a record file, which lists the dataset, split, features, seed, machine and elapsed time. The user-day table was rebuilt on 29 October 2026, and the main and baseline comparisons were rerun on it. Every detection measure of the 21 October run came back the same to four decimals, and only elapsed times differed. The comparison of the framework's parts and the rolling-feature comparison ran on 8 September 2026, the day after the data pipeline was first tested. The main comparison, the baselines and the three studies ran on 21 and 22 October 2026, and the main comparison, the baselines and the history study ran again on 29 October.

Table 3: Default settings.

Parameter	**Description**	**Default**
Scope	Reference frame for deviations	`personal`
Statistic	Deviation form	z-score
Anomaly Scorer	Anomaly model on deviations	One-Class SVM (RBF kernel, $\gamma = 1$ over the number of features times their variance, the library's scale rule)
Contamination / ν	Expected share of unusual rows	0.02
Cold-start threshold	Days before a personal profile is trusted	14
Training sample	Training rows for every anomaly model	50 000 (seeded)
Isolation Forest	Trees	200
LOF	Neighbours	25
Random Forest	Trees, depth	300, unlimited
SMOTE	Neighbours	$\min(5, n_{\text{pos}}-1)$, 5 here since $n_{\text{pos}} = 6$ malicious training rows
Alert budget	Share of test rows flagged	0.1% (403 alerts)
Low-and-slow threshold	Span of malicious activity	> 21 days
Seed	All random choices	42 (ten in the seed study)

Choosing the settings. The anomaly algorithm and the feature set were chosen by experiment, as the research proposal requires, and they came from the 45-detector comparison in Section 6.4. That comparison ran on a fixed sample of 400 users, which keeps every role's share and holds all five insiders. It was used because it could be built on the laptop before the full-population pipeline existed. The ν, γ and SMOTE neighbour settings are the defaults of the software library. The author fixed the tree and neighbour counts

in Table 3 before the experiments, and they were not searched over. The rest follow from the data. The contamination rate is the share of training rows the anomaly model is told to treat as unusual, and it is 0.02, which is inside the range of alert budgets swept, 0.01 to 5 percent. The 14-day cold-start threshold is 14 active days, about three weeks of work, and SQ3 tests it directly. The framework was first built with the Isolation Forest as its scorer, and the One-Class SVM replaced it after that comparison. The main results used 14, which never triggered, while SQ3 produces a recommendation of 60.

3.7 Testing the Code

The code carries 72 automated tests. Sixty-two run in seconds without CERT data, while ten slow tests recompute features from the 93 GB of raw logs and check them against the user-day table. Twenty-seven of the fast tests cover the model and the evaluation. They check that the profile stores the percentiles the research proposal specifies, that each scope uses its own reference, and that a never-seen event is a deviation and not a division by zero. They check that ranking uses the model's own score, and that exactly the budgeted number of rows is flagged even when scores are tied. They check that rolling features use only past days, that profiles are built from training rows only, and that the framework at population scope reduces to the population detector. Twenty-five cover the data pipeline, and these include that reading in pieces gives the same result as one pass, the removable-media flag and external-recipient detection, and labels restricted to r6.2 and to the named insiders. Ten cover the follow-up studies, and these include a hand-computed Holm correction, history shortening and an answer-key replay that recovers exactly the features the events moved. One more defect, found while writing the follow-up studies, was that SMOTE was always seeded with 42 regardless of the run's seed. This was fixed before the seed study ran, and the seed-42 results were unchanged.

4 Background

4.1 Insider Threats and the Dataset

An insider threat is a person who misuses allowed access to harm the organisation [2], and the person may be a current or former employee, contractor or partner. Real labelled insider data is rarely released [3], so research relies on synthetic data, which is made by a program to look like a real organisation's logs. Most recent studies use the CERT Insider Threat Test Dataset [9, 13], which the CERT Division of the Software Engineering Institute at Carnegie Mellon University made with help from ExactData, a data-generation company. The data follows models of how employees of a made-up organisation behave, and the authors then wrote malicious scenarios into the logs [10].

This report uses release r6.2, which covers 4 000 employees. It has five event logs, which record logon and logoff, removable-device connect and disconnect, file operations, web browsing and email. The logs run from 2 January 2010 to 1 June 2011 without a break, which is 516 calendar days. Every line is one event, stamped to the second with its user and computer (Table 5). The dataset also holds eighteen monthly snapshots of the organisation's employee directory, which run from December 2009 to May 2011. The directory is a Lightweight Directory Access Protocol (LDAP) directory, and each snapshot records each employee's role, department and supervisor, and there are 46 distinct roles. An answer key lists every malicious log line, and Table 4 summarises the five scenarios it records, one insider each.

Malicious days are very rare. Only 44 of 1 394 010 user-days are malicious, which is about one in 31 700, and the insiders are also not spread evenly across the calendar (Section 3.1).

4.2 Behavioural Baselines

A behavioural baseline describes what normal activity looks like, and a new observation is compared against it. Take a feature x_f seen on one user-day, where its reference has mean μ_f and standard deviation σ_f. The standard score, or z-score, against that reference is

$$z_f = \frac{x_f - \mu_f}{\sigma_f}.$$

It says how many spreads above or below the usual value today lies. The choice of reference is called the reference frame, and that choice is the subject of this research. A *population* baseline estimates μ_f and σ_f over every

Table 4: The five r6.2 insiders, as recorded in the official answer key. "Events" is the number of malicious log lines belonging to the insider, "Days" counts distinct malicious user-days, and "Span" is the number of whole days elapsed between the first malicious event and the last. Elec. is Electrical.

Scenario	Insider	Role	Malicious activity	Events	Days	Span	Split
1	ACM2278	Salesman	After-hours logons, removable drive, uploads to wikileaks.org	22	3	5 d	train
2	CMP2946	Salesman	Job-site browsing, job-search email to competitors, thumb-drive use	242	29	56 d	test
3	PLJ1771	ITAdmin	Keylogger web visits and download, removable drive, after-hours logons, an email that reached 24 colleagues	18	2	1 d	train
4	CDE1846	Elec. Engineer	Logons to other users' PCs, file opens, documents emailed to a personal address	134	9	63 d	test
5	MBG3183	Elec. Engineer	Four uploads to a file-sharing site in one minute	4	1	0 d	train

employee, a *role* baseline estimates them over the employee's role group, and a *personal* baseline estimates them over the employee's own history.

Behavioural counts are mostly zero with a few large values. In r6.2, 80.3% of users never write to removable media, and across all user-days, 90.1% have no such write. A single large day inflates σ_f and hides the next one. The median alternative is less affected by extreme values, because it swaps the mean for the median, which is the middle value, and swaps the standard deviation for the interquartile range (IQR). The IQR is the spread of the middle half of the values, and it is scaled so that it matches σ for bell-shaped

Table 5: The r6.2 logs as extracted. Sizes are in binary units, and event counts are line counts of the extracted files, less the header line.

Log	Events	Size	What is drawn from it
logon	3 530 285	230 MB	Logons, logoffs, after-hours logons
device	1 551 828	133 MB	Removable-device connections
file	2 014 883	1.24 GB	File operations, copies to removable media
http	117 025 216	84.0 GB	Web requests, uploads, distinct hosts
email	10 994 957	7.53 GB	Emails sent, external recipients, attachments, size
LDAP	18 snapshots		Role, department, supervisor, company domain
Total	135 117 169	93.2 GB	

data [14]:

$$z_f^{\text{median}} = \frac{x_f - \text{median}_f}{\text{IQR}_f / 1.349}.$$

4.3 Anomaly Detection Without Labels

An anomaly is an observation far from what is usual. Anomaly detectors find such points without labels, and they learn only from data that is mostly normal. Labelled insider activity is scarce, so they suit data with few labels [15].

Isolation Forest [16] isolates points with random splits, and it splits on one feature at a time, over and over. An unusual point is cut off from the rest in fewer splits, so the average number of splits needed to isolate a point is its score, which is flipped so that higher means more unusual. Its training time grows about in step with the number of rows, and it also copes with features that repeat the same information, because each split looks at one feature [16].

Local Outlier Factor (LOF) [17] measures how crowded the space around a point is, and compares this with the crowding around the point's

nearest neighbours, 25 here. A point in a much emptier region than its neighbours scores high.

One-Class Support Vector Machine (SVM) [18] fits a boundary between the data and the origin. It measures how alike two points are with a function called a kernel, $k(\cdot, \cdot)$, and then picks the straight boundary that separates the data from the origin with the widest gap. A setting ν between 0 and 1 limits the share of training points allowed outside the boundary. The score is the signed distance from the boundary, $\sum_i \alpha_i k(x_i, x) - \rho$. The x_i are the training points the model keeps, which are called its support vectors, and the α_i are weights set in training, and so is the constant ρ. A radial basis function (RBF) kernel is $k(x, x') = \exp(-\gamma \|x - x'\|^2)$, where γ sets how fast similarity falls with distance. With this kernel the model is sensitive to the shape of the whole feature space, which matters when features are added. Far from every support vector the kernel sum is zero, so the score then equals the constant ρ.

4.4 Supervised Detectors and Rare Classes

A supervised classifier learns to tell malicious from benign directly from labelled examples. On the CERT data, Noever compared 88 algorithms from 16 families, and Random Forests came out strongest [4]. A Random Forest is a group of many decision trees that vote.

Malicious activity is 0.003% of user-days, so the two classes are extremely unequal in size, and the research proposal therefore specified rebalancing the training set. The Synthetic Minority Over-sampling Technique (SMOTE) makes new malicious examples, and each new point sits between a real malicious example and one of its k nearest malicious neighbours [19]. SMOTE is applied to the training part only, after the split, so no made-up row can then carry information from the test window.

4.5 Measuring Detection of Rare Events

ROC-AUC can mislead when malicious days are this rare. ROC-AUC is the area under the receiver operating characteristic curve. It is the chance that a random malicious day ranks above a random benign day, where the malicious days are the positives and the benign days the negatives. In the test window, benign days outnumber malicious ones about 10 600 to one. A detector can then order the benign mass well and score a high ROC-AUC with no malicious day anywhere near the top of the queue [20, 21]. Average precision (AP) is used instead. It is the precision averaged over every cut-off at which one more malicious day is found, and its chance level equals the share of malicious days, here about 10^{-4}. It is decided mostly by the top of the ranking, which is the only part an analyst ever reads.

The detectors are compared at a fixed alert budget. A Random Forest gives a probability, an Isolation Forest gives a split count and a One-Class SVM gives a distance, so cutting all three at one value would compare a probability with two numbers that have no shared meaning. So each detector may flag the top b fraction of test user-days. This is the *alert budget*, and it also reflects the assumption that an analyst team can investigate only a fixed number of alerts a day.

At a budget, three measures are well defined for every detector. Recall is the share of malicious days flagged, precision is the share of alerts that are malicious, and F1 combines the two in a single number, which is low when either is low. *Lift* is the precision divided by the share of malicious days, and it says how many times better than random the alerts are.

4.6 Explanations and Statistical Tests

A flagged user-day with no reason attached must be investigated from scratch. SHAP (SHapley Additive exPlanations) splits a model's output among its input features, and it does this with Shapley values. These come from game theory, and they are a rule for sharing a result fairly among the things that produced it. The feature contributions add up to the difference between this prediction and the average prediction [22]. TreeExplainer computes these values exactly and quickly for groups of trees [23]. The per-user framework has a more direct explanation available, which is the deviation vector itself, and its largest entries name the features on which today departs most from this user's normal.

Several parts of every detector involve chance, and these include sampling, building forests, SMOTE and tie-breaking. A difference seen under one seed may just be that one draw. Re-running each detector under n

seeds gives n paired results. The Wilcoxon signed-rank test [24] ranks the paired differences by size and then tests whether the positive and negative ones balance. It does not assume a bell-shaped distribution, and it is the test recommended for comparing classifiers over paired runs [25]. The result is a p-value, which is the chance of a difference at least this large if there were no real difference. The test is two-sided, which means a difference in either direction counts. Take $n = 10$ seeds with every difference in the same direction. The exact two-sided p-value is then $2/2^{10} \approx 0.002$, which is the smallest the test can give at this n.

When m tests are run together, the chance that at least one is a false finding grows. The Holm-Bonferroni procedure [26] keeps that chance at the chosen level α, 5 percent here, and this report calls it the Holm correction. It compares the i-th smallest p-value with $\alpha/(m - i + 1)$ and stops at the first failure.

5 Related Work

Machine learning on the CERT data. Le et al. showed that the unit of counting changes what detectors find [8], where the unit may be the user-day, the user-week or the session. Yuan et al. replaced hand-made features with a long short-term memory network (LSTM), which is a neural network for sequences, and a convolutional neural network (CNN) classifier then follows it [27]. These, like the classifiers Noever compared, train one model on the data of every user. This work keeps the user-day unit and Noever's Random Forest as its comparison point, and changes only the reference frame.

Per-user and user-centred modelling. Tuor et al. ran deep neural networks on a stream of CERT r6.2 events, and these networks learn without labels and split each score into per-feature contributions. They reported gains over Isolation Forest, principal component analysis (PCA) and SVM baselines [7]. Commercial UEBA platforms such as Microsoft Sentinel take the same idea into production, where they profile each user against their own history and their peers' [6]. Both need a large neural network trained on streaming data or a licensed SIEM and analytics platform, which a resource-constrained organisation by definition lacks. The framework here pursues the same idea with simple summary statistics and one standard anomaly model, on exported log files. Alahmadi et al. interviewed security operations centre analysts and found false positives to be the dominant complaint, and analysts come to distrust tools that raise them [5]. This is why this report measures *who* an alert budget is spent on.

Explanations and forensic readiness. None of the works above scores its explanations against the answer key, but this report does. Shoderu et al. argue that insider-threat defence in software-as-a-service (SaaS) environments remains reactive, so they propose building in digital forensic readiness, which means capturing evidence early, in a form that holds up legally. They follow the ISO/IEC 27043 standard [28]. A per-user baseline explains each alert against the user's own history, and that is one source of such evidence.

Gap. In the works reviewed above, the per-user approach exists only as large neural networks or commercial platforms, and none of them combines three things. The first is a lightweight per-user baseline built on exported log files and consumer hardware, and the second is a controlled comparison with the population practice it would replace. The third is scoring its detection and explanations against the answer key.

6 Discussion

This section uses one main setting unless it says otherwise. The population is all 4 000 users, the main seed is 42, and a seed is the starting number for every random choice. The alert budget is 0.1%, which means 403 alerts over 403 245 test user-days, where a user-day is one user on one day. Those days hold 38 malicious days from two insiders, and Section 6.5 shows that the conclusions hold across ten seeds.

6.1 Main Comparison

Table 6 compares the framework with every population detector and both floors.

The framework does better than the Random Forest on each point below. Recall is the share of malicious days a detector flags. The framework flags 21 of the 38 malicious test days, a recall of 0.553, while the Random Forest flags 8, a recall of 0.211, and only the framework finds *both* test-window insiders. Its alerts are 553 times more likely to be malicious than a random alert. It spends 3.5% of its alert queue on IT administrators, while the Random Forest spends 22.6%. The framework trains in about eight seconds, and the Random Forest takes about five minutes (Section 6.7).

The gain comes from the reference frame, which is what a user's day is compared against. The same-algorithm control of Section 3.3 shows this. The control is `pop-ocsvm`, which differs from the framework only in the reference frame, and it finds 3 malicious days while spending 20.8% of its alerts on administrators. So changing only the reference frame raises recall from 0.079 to 0.553 and cuts the high-privilege alert share to about one sixth.

The ordering does not depend on the chosen budget, which Figure 3 shows. From 0.05% upwards the framework's recall beats every population detector, and at 0.5%, a budget of about 13 alerts a day, it reaches 0.84. The Random Forest stays at 0.21 from 0.05% to 1% because only 304 of its rows have a non-zero probability, and past those rows its ranking runs out, so 85% of its 0.5% budget is a random draw among ties. At the smallest budget of 40 alerts the framework finds nothing, and Section 6.10 traces this to a ceiling in the One-Class SVM's scores.

Table 6: Full population, 0.1% alert budget of 403 alerts, seed 42. "Days" is how many of the 38 malicious test days were flagged. "HP share" is the share of alerts spent on high-privilege users, which are the users in the ITAdmin role, and they make up 2.43% of test user-days. The best value in each column is in bold. The role variant has the highest AP because AP is decided by the very top of the queue (Section 6.10). A ROC-AUC below 0.5 means worse than random ordering. Lift is precision divided by the share of malicious days, and it is 553 for the framework and 211 for the Random Forest. Three variants are not listed here, the median-statistic, size-channel and role-scoped Isolation Forest variants, and their numbers appear in Section 6.3. Training cost is reported in Section 6.7.

Detector	**AP**	**ROC-AUC**	**Days**	**Insiders**	**HP share**
Per-user framework					
`peruser-ocsvm-personal-zscore`	0.0383	**0.967**	**21**	**2/2**	3.5%
`peruser-ocsvm-role-zscore`	**0.0433**	0.675	3	1/2	0.7%
`peruser-iforest-personal-zscore`	0.0030	0.921	3	1/2	7.7%
Population, unsupervised (same algorithms, population reference)					
`pop-ocsvm`	0.0364	0.446	3	1/2	20.8%
`pop-iforest`	0.0009	0.935	0	0/2	49.4%
`pop-lof`	0.0003	0.831	0	0/2	**0.0%**
Population, supervised (SMOTE-balanced)					
`rf` (proposal baseline)	0.0077	0.605	8	1/2	22.6%
`lr`	0.0284	0.864	2	1/2	9.7%
`gb`	0.0003	0.379	1	1/2	44.4%
`dt`	0.0001	0.500	0	0/2	2.7%
Floors					
`volume`	0.0284	0.821	2	1/2	4.0%
`random`	0.0001	0.445	0	0/2	2.0%

6.2 SQ1 Finding Slow Insiders

The framework finds far more low-and-slow days than the other detectors. Every malicious day in the test window belongs to a low-and-slow insider, so the main recall answers SQ1 directly. The per-user baseline finds 21 of these days while the population Random Forest finds 8, so the baseline finds 2.6 times as many. The same algorithm with a population reference finds 3, so

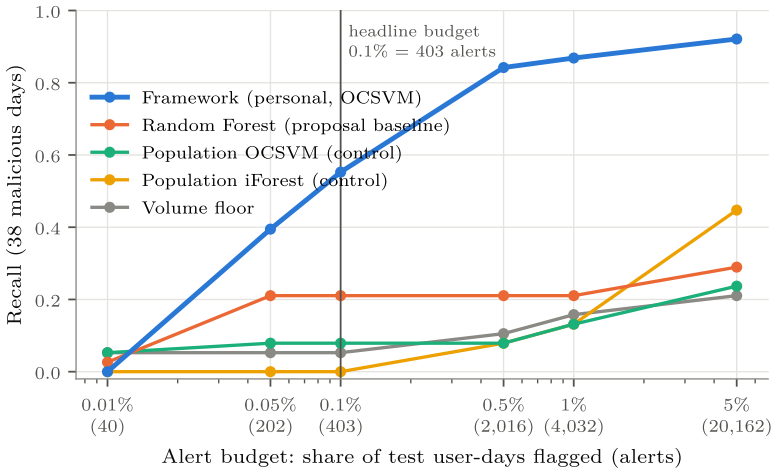


Figure 3: Recall of the 38 malicious test days at alert budgets from 40 to 20 162 alerts, full population, seed 42.

the baseline finds 7 times as many.

The framework's gain lies in days that are odd for the person but normal for the organisation, which Figure 4 shows. CMP2946's scenario is job-hunting and then copying data to a thumb drive, and it never produces a day that is extreme for the organisation. This user made exactly 162 web requests on each of their 249 training days, and in the test window they made about 164 a day on average. Across 4 000 users that is ordinary, but against their own history the scenario's days stand out sharply. The alert line is the queue position where the 0.1% budget is cut, and the framework puts 18 of their 29 malicious days inside the alert line while the population-referenced model puts none there. CDE1846 is different because their malicious days are large bursts of emailed attachments, which are extreme for anyone. Both reference frames catch the same 3 of their 9 days, 22 March, 11 April and 25 April 2011, so the per-user baseline neither gains nor loses on them. This is exactly where the research question placed the advantage, in behaviour that is unusual for the person but not for the population.

Alerts ahead and delay. Only the framework ranks both insiders near the top of its queue. Table 7 gives two measures for each insider across all ten seeds of Section 6.5. The first is how many user-days rank above the insider's best malicious day, and the second is how many days passed from their first malicious day to their first flag. Every population detector buries one of the two insiders thousands of alerts deep. The Random Forest ranks CMP2946's scenario highly but puts CDE1846 about 24 000 alerts down, which is roughly

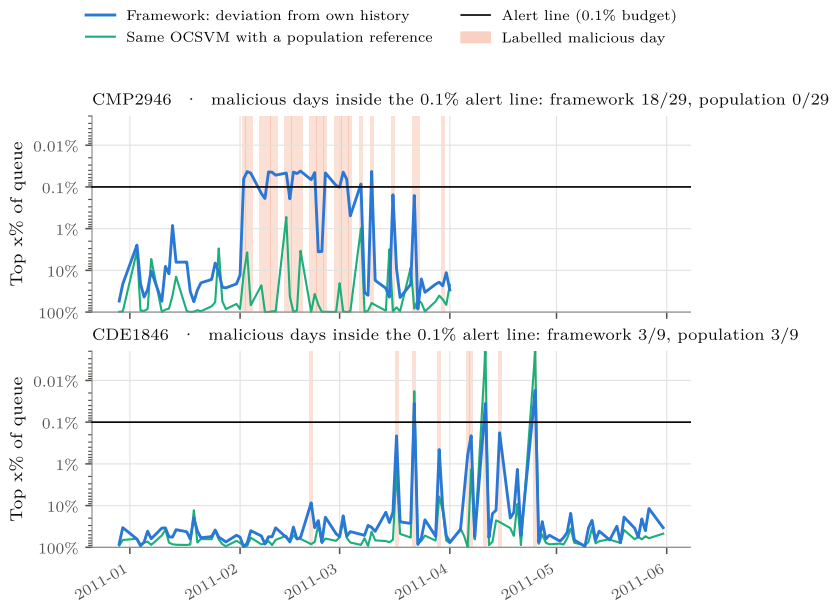


Figure 4: Daily queue position of the two low-and-slow insiders over the test window, full population, seed 42, shown under the framework and under the same One-Class SVM with a population reference. Shaded days are labelled malicious. The black line is the alert line, which is the queue position where the 0.1% alert budget is cut.

sixty times its budget, so it never flags them. The same-algorithm control and the volume floor do the reverse. The framework brings both insiders inside its first 200 alerts in every seed. It flagged CMP2946 on the first day of their scenario, and it flagged CDE1846 29 days in, on 22 March 2011. Three other detectors first reached CDE1846 on that same day, taking the median over seeds, and they are the role-scoped variant, the Isolation Forest variant and the population One-Class SVM, while the volume floor reached them 20 days later.

Table 7: How far down each detector's queue an analyst must read to reach each insider. Alerts ahead is the number of user-days ranked above the insider's best malicious day, given as the median over ten seeds, with the range where it varies. Detection delay is the median number of days from the insider's first malicious day to the first flagged one, at the 0.1% budget. An alerts-ahead value of 0 means the insider's best day is at the top of the queue, and "never" means not flagged in any seed. Display names map to code names as follows. Framework, personal, OCSVM is `peruser-ocsvm-personal-zscore`, role scope is `peruser-ocsvm-role-zscore`. iForest scorer is `peruser-iforest-personal-zscore`, and the population OCSVM and iForest are `pop-ocsvm` and `pop-iforest`.

	Alerts ahead		Detection delay (days)	
Detector	**CMP2946**	**CDE1846**	**CMP2946**	**CDE1846**
Framework (personal, OCSVM)	186 (168 to 191)	85 (6 to 140)	0	29
Framework, role scope	5 626 (5 108 to 6 479)	0	never	29
Framework, iForest scorer	1 239 (726 to 2 398)	80 (40 to 218)	never	29
Population OCSVM (control)	2 175 (1 976 to 2 717)	0 (0 to 1)	never	29
Population iForest	3 595 (2 754 to 6 083)	1 414 (685 to 1 951)	never	never
Random Forest (proposal's comparison)	45 (19 to 78)	24 367 (9 093 to 77 971)	0	never
Volume floor	30 040	0	never	49

6.3 SQ2 False Alerts on Administrators

Population detectors waste many alerts on administrators, but the framework does not. High-privilege users are 2.43% of the test user-days, and Figure 5 shows how much of each queue they take up. Four population detectors over-spend on them by nine to twenty times. `pop-iforest` sends 49.4% of its alerts to administrators doing their jobs, `gb` sends 44.4%, the Random Forest sends 22.6% and `pop-ocsvm` 20.8%. Every per-user variant without the size channel sits between 0.5% and 16.1%. The lowest is the One-Class SVM on the median statistic at 0.5%, next is the role-scoped One-Class SVM at 0.7%, and the role-scoped Isolation Forest is at 1.7%. The same result can be read as a false positive rate, which is the share of innocent days that get a false alert. The Random Forest raises a false alert on 0.93% of the 9 817 administrator test days while the framework does so on 0.14%, a 6.5-fold reduction.

The reference frame removes this bias, not the algorithm, and holding

the algorithm fixed shows this. Isolation Forest with a population reference spends 49.4% of its alerts on administrators, but with a personal reference it spends 7.7%, and One-Class SVM goes from 20.8% to 3.5%.

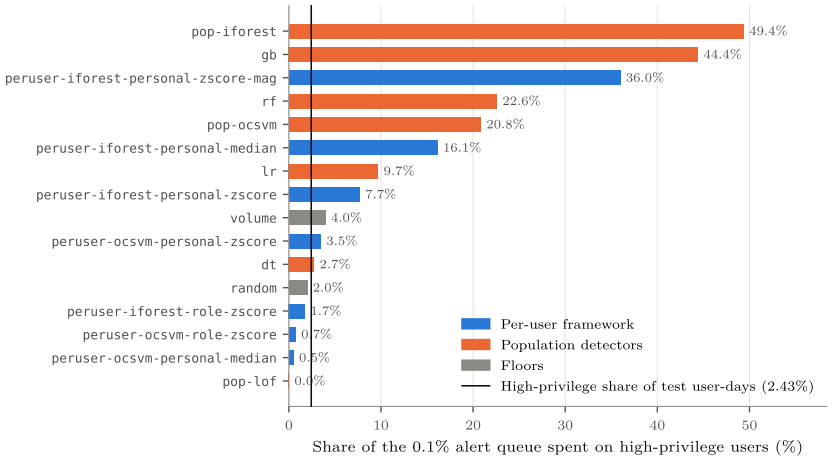


Figure 5: Share of each detector's 403 alerts spent on high-privilege users, full population, seed 42. The vertical line is their share of the test population.

The problem is not confined to the role the research proposal anticipated, because executives are also unusual for good reason. The population One-Class SVM raises a false alert on 18.7% of the test days of the user in the President role and on 4.0% of the test days in the Vice-President role, while Isolation Forest's rates are 15.9% and 2.9% and the framework's rates are 0.9% and 0%. A detector that only special-cases administrator roles would miss this, but one that measures each user against themselves needs no list.

Two results limit the claim. First, the lowest false-alert options find fewer insider days. The role-scoped framework has one of the lowest administrator shares at 0.7% but finds only 3 malicious days, and the median-statistic variant is lower still at 0.5% but finds only one. So the personal scope with z-scores is the better trade-off, but it is not the lowest false-positive option. Second, the size channel raises Isolation Forest's administrator share from 7.7% to 36.0%, which brings back the population bias the framework exists to remove, so the framework does not use it.

6.4 Comparing the Parts of the Framework

The research proposal requires the anomaly algorithm to be chosen by experiment. Table 8 sums up the 45-detector comparison, which tries every combination of scope, statistic, algorithm and the size channel. That gives 36 framework variants, and they run alongside the 9 other detectors. The run uses the 400-user sample, which has 40 360 test user-days and 40 alerts at 0.1%. The table shows the z-score cells and two variant rows, and the rest follow the same pattern.

Table 8: Comparison of the framework's parts on the 400-user sample, seed 42. Each cell gives average precision / recall at 0.1%, that is 40 alerts, with the z-score statistic, and a recall of 0.08 is 3 of the 38 malicious days. The population-scope cells for One-Class SVM and Isolation Forest exactly match `pop-ocsvm` and `pop-iforest` run on the same 400-user sample, and those runs are not listed. The LOF cell agrees with `pop-lof` to three decimals, 0.00186 against 0.00187, which confirms that the three scopes share one piece of code.

Anomaly Scorer	**Population**	**Role**	**Personal**
One-Class SVM	0.046 / 0.08	0.086 / 0.08	**0.292 / 0.50**
Isolation Forest	0.009 / 0.00	0.022 / 0.08	0.059 / 0.08
LOF	0.002 / 0.00	0.002 / 0.00	0.027 / 0.00
Variants of the personal scope			
Median statistic (OCSVM / iForest)	0.030 / 0.03	and	0.044 / 0.08
+ size channel (OCSVM / iForest)	0.173 / 0.37	and	0.047 / 0.08

Three conclusions follow. First, the reference frame matters for every algorithm. The personal scope gives the highest average precision in all three rows, and it is six to fifteen times that of the population scope, where the ratios are 6.4, 6.8 and 14.6. Second, the algorithm also matters. One-Class SVM on personal z-scores is best by five times, LOF finds nothing in any setting and is also the slowest model, and Isolation Forest sits in between. Third, the median statistic did not help the best setting. Slow drift is a gradual change in a user's behaviour, and the median statistic was expected to resist spikes and so show slow drift better. A likely reason it did not help is that counts that are zero on most days have an IQR of zero, and raising that scale to one throws away the user's real variation.

Rolling features. Rolling features help one scorer and hurt another. A rolling mean is the average of a feature over the previous 7 or 30 days, and adding 7-day and 30-day rolling means of every feature gives 39 features in all. This part was rerun on the full population (Table 9). The Isolation Forest looks at one feature per split, so it copes with repeated information. With the median statistic, the variant in Table 9, its average precision gains nine times, and with z-scores it gains 3.5 times, from 0.0030 to 0.0106. One-Class SVM loses most of its recall. A likely reason, not tested here, is that thirteen features become 39 that move together, and distances in the RBF kernel then lose their meaning. The population detectors gain nothing while the role-scoped One-Class SVM gains one day, and a likely reading, not tested further, is that drift carries information mainly against a person's own baseline. So the algorithm and the feature set must be chosen together, and the best setting overall is One-Class SVM on personal z-scores with the base features.

Table 9: Effect of the rolling-window features on the full population, seed 42, 0.1% budget. The base column is the 29 October 2026 run and the rolling column is the 8 September 2026 run, and the three main runs gave the same detection values.

	Average precision		**Recall at 0.1%**	
Detector	**base**	**+ rolling**	**base**	**+ rolling**
Framework, iForest, median statistic	0.0041	**0.0378**	0.079	0.079
Framework, iForest scorer	0.0030	0.0106	0.079	0.079
Framework (personal, OCSVM)	**0.0383**	0.0125	**0.553**	0.053
Framework, role scope	0.0433	0.0387	0.079	0.105
Population OCSVM	0.0364	0.0280	0.079	0.079
Volume floor	0.0284	0.0281	0.053	0.053
Population iForest	0.0009	0.0006	0.000	0.000

6.5 Consistency Across Random Seeds

The key results are now repeated under ten seeds. Every comparison so far used one seed, so Table 10 and Figure 6 repeat the seven key detectors under ten seeds, where every random choice is re-drawn each time.

Table 10: Mean ± standard deviation over ten seeds, full population, 0.1% budget. "Days" is the range of malicious days flagged across the seeds, and "Fit" is the mean fit time in seconds, that is the training time.

Detector	AP	Recall	Days	Insiders	HP share	Fit (s)
`peruser-ocsvm-personal-zscore`	0.0363 ± 0.0011	**0.547 ± 0.011**	**20 to 21**	**2/2**	3.4 ± 0.2%	7.9
`peruser-ocsvm-role-zscore`	**0.0473 ± 0.0049**	0.079 ± 0.000	3	1/2	**0.5 ± 0.1%**	6.6
`peruser-iforest-personal-zscore`	0.0029 ± 0.0008	0.076 ± 0.008	2 to 3	1/2	7.4 ± 1.5%	7.4
`pop-ocsvm`	0.0360 ± 0.0063	0.079 ± 0.000	3	1/2	20.4 ± 1.7%	2.6
`pop-iforest`	0.0009 ± 0.0001	0.000 ± 0.000	0	0/2	47.5 ± 7.3%	3.3
`rf`	0.0049 ± 0.0027	0.163 ± 0.046	3 to 8	1/2	21.3 ± 2.5%	306.1
`volume`	0.0284 ± 0.0000	0.053 ± 0.000	2	1/2	4.0 ± 0.0%	0.0

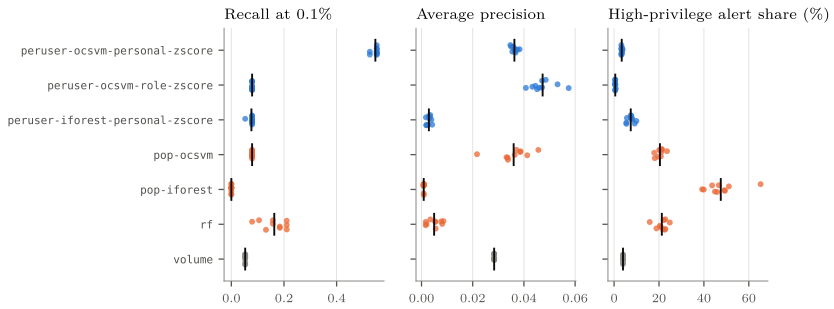


Figure 6: Each detector under ten seeds, where dots are seeds and the black tick is the mean.

The framework's result is stable across seeds. It found 20 or 21 of the 38 malicious days in every seed, and it found both insiders in every seed. The Random Forest ranged from 3 to 8 days, so its 8 days in the main single-seed comparison was its best draw, and for this reason the ten-seed means in Table 10 differ slightly from the seed-42 values in Table 6.

The paired tests confirm the picture. A paired test asks whether one detector beats another across the same seeds. There are eighteen framework-versus-detector comparisons, and seventeen went the same way in all ten seeds. Fifteen favoured the framework, two went against it, both against the role variant, and one was mixed, average precision against `pop-ocsvm`. Each

of the seventeen gives an exact two-sided Wilcoxon $p = 0.002$, and after correcting for all eighteen, $p_{\text{Holm}} = 0.035$, where 0.035 is 0.002 times 18. This is the smallest adjusted p-value a ten-seed design with eighteen tests can give. In the rest of this report, significant means a Holm-corrected p under 0.05. The framework has significantly higher recall than every other detector. Its high-privilege alert share is significantly lower than every population detector's, and it is also significantly lower than its own Isolation Forest variant's and the volume floor's. The volume floor's share never varies, so a steady 0.6 point gap passes the test. The framework also has significantly higher average precision than the Random Forest, both Isolation Forest detectors and the volume floor.

Three comparisons do not favour the framework, and they limit what is claimed. Against its same-algorithm control, average precision is a tie, with five seeds going each way and $p = 1$. Both detectors give one of CDE1846's attachment-burst days the top score. For the framework, 140 rows share that score, as Section 6.10 explains, so its position in Table 7 is set by the tie-break. That one day decides most of the average precision, so the personal reference frame's edge is in recall and in where the second insider lands, not in the top-ranked alert. Against the role-scoped variant, the framework loses in all ten seeds on both average precision and administrator alert share, but it finds seven times as many malicious days. The role reference is the stronger way to cut false alerts, while the personal reference is the stronger detector.

6.6 SQ3 How Much History Is Needed

A personal baseline becomes stable at about 30 days. Figure 7 and Table 11 show the history-length study, and the two views of stability agree. With three days of history, a third of all benign test days show some feature beyond $|z| = 3$, because a few days cannot estimate a user's spread. That share falls to 12.9% at 14 days and 8.7% at 30 days, and it then flattens, and with the full history it reaches 5.9%. The per-user means settle in the same way, and by 30 days 99.5% of user-feature means lie within half a standard deviation of their full-history value.

Detection needs more history than stability does. At 30 days and below, every alert in the budget lies in the group of days tied at the top score, and the tie fraction is 1.0 in every seed. This is the ceiling of Section 6.10, so the recall there is a tie-break draw. The personal baseline finds almost nothing, and it is worse than simply using the role baseline, whose recall is 0.079. At 60 active days, about three months of working days, recall jumps to 0.34 and both insiders are found. Gains continue to roughly 120 to 180 days and

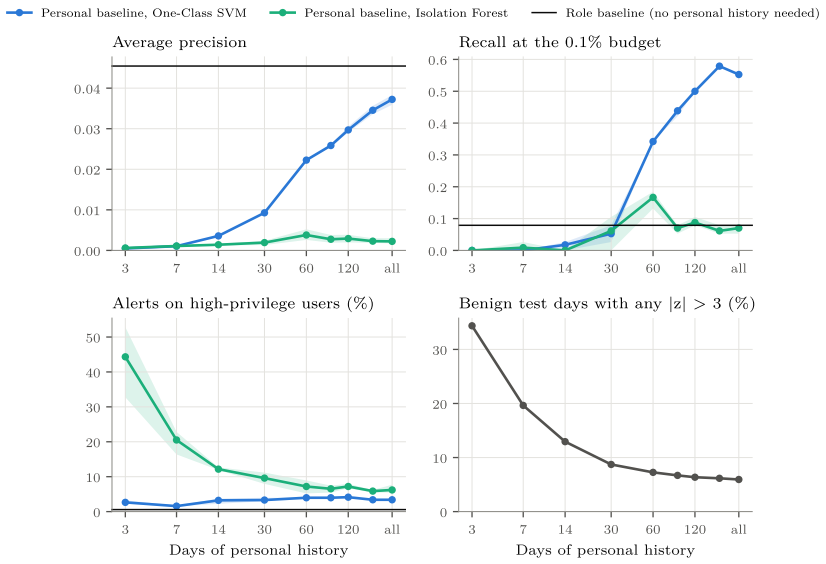


Figure 7: Detection and stability against days of personal history, where history means the most recent active days before monitoring starts. Lines are means over three seeds and bands span the seeds.

Table 11: History-length study, One-Class SVM personal baseline, means over three seeds. Because they are means, insider-level recall can take values other than 0, 0.5 and 1. The role row is also a three-seed mean, which is why it differs from Tables 6 and 10. "False-deviation days" is the share of benign test days with any feature beyond $|z| = 3$.

History	AP	Recall at 0.1%	Insider-level recall	HP share	False-deviation days
3 days	0.0005	0.000	0.00	2.7%	34.4%
7 days	0.0010	0.000	0.00	1.6%	19.6%
14 days	0.0036	0.018	0.33	3.2%	12.9%
30 days	0.0093	0.053	0.50	3.3%	8.7%
60 days	0.0223	0.342	1.00	4.0%	7.3%
90 days	0.0259	0.439	1.00	4.0%	6.7%
120 days	0.0297	0.500	1.00	4.1%	6.4%
180 days	0.0345	0.579	1.00	3.4%	6.2%
All (248)	0.0372	0.553	1.00	3.4%	5.9%
Role baseline	0.0454	0.079	0.50	0.6%	

then level off. One caution applies to the shortest histories. Below 13 days the anomaly model has fewer than its usual 50 000 training rows, because 4 000 users times 13 days is just above 50 000, so the shortest histories mix profile quality with training-set size. From 14 days upward the training size is constant, so from there the trend is due to the profiles alone.

The answer to SQ3 is about 30 active days for a *stable* baseline and about 60 for a *useful* one. Too little history is not just useless, because for the wrong scorer it is harmful. With three days of history the Isolation Forest variant spends 44% of its alerts on administrators (Figure 7, bottom left). An under-estimated spread makes naturally variable users look unusual, so a short personal baseline repeats the population failure. The One-Class SVM variant is unaffected, and it stays between 1.6 and 4.1% throughout.

Why 14 days is too short. A 14-day profile does not yet describe a person's normal. Take the user-feature pairs that vary over the full history. At 14 days, 6.8% of them have a standard deviation of exactly zero, because the user showed no variation in that feature within those 14 active days, and the scale is then floored at one. At 60 days this share falls to 2.3%. At 14 days, one in ten per-user means still sits more than 0.4 standard deviations from its full-history value, so a 14-day profile describes about three weeks of behaviour, not the person's normal. Against it, 12.9% of ordinary days are scored as deviations, which is more than twice the 5.9% of the full history. The practical reading is that an employee should not be judged against their own first three weeks, when a routine is still forming. This reading is an interpretation, because the study keeps each user's most recent N active days, so it imitates a short logging history, not a new employee. The measurements show that the 14-day cold-start threshold is too low. Until about 60 days are available, the role baseline is both safer and more effective. It needs no personal history, it is safer with a 0.6% administrator share, and it is more effective with recall 0.079 against 0.018. Raising the threshold has a cost that the main experiment hides, because the 14-day threshold never fell back. Under a 60-day threshold, 43 of the 4 000 users would have been scored against their role instead, which is 1.1%, counted from the training window.

6.7 SQ4 Computing Cost

Table 12 reports the measured cost on the laptop. The per-detector rows are means over the ten seeds of the seed study, because the elapsed time of a single run on a laptop proved unreliable. The main comparison was run on 21 October 2026 and again on 29 October, and both runs gave the same detection measures, but their fit times differed by up to two times, depending on whatever else the laptop was doing. The framework took 7.4 seconds in one run and 15.8 in the other, while the Random Forest took 289 seconds and then 544. The reliable quantity is the ratio between the two detectors, which was 39 in the seed study and 34 to 39 in the two single runs.

Table 12: Computational cost on the full population of 990 765 training and 403 245 test user-days. All three per-detector columns are ten-seed means, and fit times carry their standard deviation. "ms/row" is the mean scoring time in milliseconds per test user-day, and "Peak" is the most memory used by fit and score together. The rows marked run once are as recorded in the pipeline's documentation when they were run, and memory was traced for the per-detector rows only.

Detector	**Fit (s)**	**ms/row**	**Peak (GB)**
Framework (personal, One-Class SVM)	7.9 ± 0.5	0.027	1.3
Framework, role scope	6.6 ± 0.5	0.025	0.9
Framework, Isolation Forest scorer	7.4 ± 0.3	0.007	1.3
Population One-Class SVM	2.6 ± 0.1	0.023	0.2
Population Isolation Forest	3.3 ± 0.3	0.005	0.2
Random Forest (proposal's comparison)	306.1 ± 21.1	0.0015	1.8
Volume floor	0.0	0.0000	0.04
Raw logs to user-day table (93 GB), 12 workers, run once	198		
The same with 4 workers, run once	900		2.8
User-day table to frozen dataset, run once	3		

The framework builds 4 000 personal profiles, 46 role profiles and the population profile from 990 765 rows, then fits the anomaly model, and the whole step takes about eight seconds. Only one anomaly model is fitted, however many users there are, so cost grows with rows and not with users. The framework takes 7.9 seconds while the same One-Class SVM with a population reference takes 2.6, and the difference is the cost of building the profiles and the deviation vectors. The Random Forest is 39 times slower to train, and SMOTE adds 990 753 made-up malicious rows to the six real ones, which doubles the training set to 1 981 518 rows. The framework is slower than the Random Forest to *score*, at 0.027 ms per user-day against 0.0015 ms, but a 4 000-person organisation produces at most 4 000 user-days a day, and these score in about a tenth of a second. The expensive step is the one-off reduction of the raw logs, which reads 93 GB in 3.3 minutes with 12 workers, while limited to 4 workers it takes 15 minutes and at most 2.8 GB of memory. These timings are noted in the pipeline's documentation. Per-user baselining is therefore feasible on consumer hardware, and it is also cheaper to train than the population detector it replaces.

6.8 SQ5 Context for the Analyst

A user study of investigation time was outside the project's reach, so SQ5 is answered with two measurable stand-ins. The first is alerts ahead, which is how far down the ranked queue an analyst must read before reaching each insider (Section 6.2). The second is reported here, and it asks whether the reason each system gives for a malicious day names what the insider actually did.

Figure 8 scores the top three features of each explanation against the answer key's ground truth for all 38 malicious test days. SHAP is scored in two readings, where the first ranks contributions by size and ignores their sign, and the second keeps only the features that push towards an alert. The framework's top reason names a feature the malicious events actually changed on 94.7% of days, while a random explanation would do so on 29.6% of days, which is the chance formula of Section 3.5 with one named feature. With the same method but a population reference the rate is 52.6%, and against the role group it is 39.5%. The explanation method is the same in all three, and only the reference frame differs. The framework actually flagged 21 malicious days, and on every one of them its top-named reason is correct. Precision@3 is the share of the three named features that are relevant, and it is 0.64 for the framework, 0.53 with a population reference, 0.44 with a role reference and 0.26 for the positive SHAP reading, while the chance rate

is 0.30.

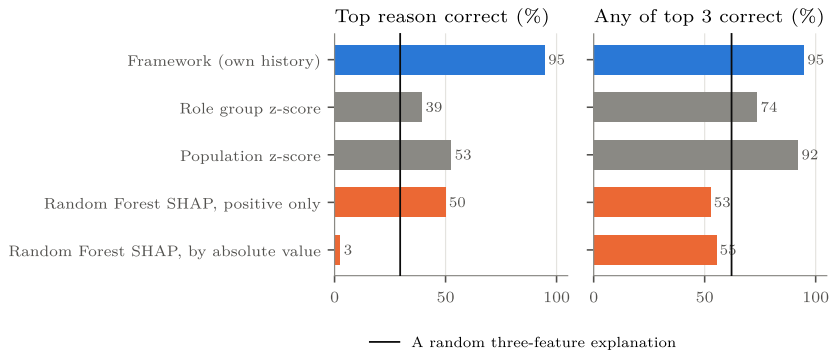


Figure 8: How often each explanation names a feature the insider's malicious events actually changed that day, over the 38 malicious test days, full population, seed 42. The vertical line in each panel is the chance rate, which is 29.6% for a random one-feature explanation on the left and 62.1% for a random three-feature explanation on the right.

How well the Random Forest's SHAP explanations perform depends on how they are read. Ranked by size ignoring sign, they name a relevant feature first on only 2.6% of malicious days, which is far below the 29.6% of a random explanation, and they name one on none of the 8 days the forest itself flagged. The forest assigns most low-and-slow days a probability near zero, so its largest SHAP values are reasons *against* an alert. Reading only the features that push towards an alert raises the rate to 50.0% over all malicious days, and on the forest's own 8 alerts it rises to 87.5%. So when the forest does raise an alert, its explanation names a relevant feature first on 7 of its 8 alerts, provided it is read for positive contributions.

Two differences from the framework remain. The first is coverage, where the framework flagged 21 malicious days and its top-named reason was right for all 21, while the forest raised 8. Across all 38 malicious days, even the positive reading of SHAP includes a relevant feature in its top three less often than a random explanation would, 52.6% against 62.1%, and the reason is that the forest learned the malicious class from the burst insiders. The second difference is context. A SHAP value is a contribution to a probability, and it says nothing about what is normal for the person, while a deviation says what the value is today, what it usually is for this user, and how far apart the two are.

The insider-level breakdown mirrors the detection result. CDE1846's days are extreme for anyone, and on those days every z-score explanation is correct.

On CMP2946's days the framework's top reason is correct 93% of the time, while a population reference is correct 38% of the time and a role reference 21%. Personal context matters for exactly the behaviour that is unusual only for the person.

Table 13 shows what this means at the analyst's desk. Take CMP2946's day on 18 February 2011, when the framework reports that web requests rose to 173. This user made exactly 162 on every training day, so the spread is zero, raised to one, and that is a deviation of 11. To the population this is a 1.8-standard-deviation day, which is ordinary among 4 000 users. Now take CDE1846's day on 25 April 2011, one of the days tied at the maximum score, where the framework reports eight file operations against a usual zero. That is 12 standard deviations, while the population reference rates it at 1.4. This is the trace of the insider opening files on other users' machines, and it sits alongside the emailed attachments that every reference frame sees. The forest flagged neither day, and its SHAP values for both are small and negative.

Table 13: What each system tells the analyst about the two insiders' highest-scored malicious days. "Usual" and "p90" are the user's own median and 90th-percentile day. Rows are the three features the framework names first for CMP2946, and the four email features and the file feature for CDE1846. A negative SHAP value is a reason against an alert.

Feature	Today	Usual	p90	z (own)	z (population)	Random Forest SHAP
CMP2946, 18 February 2011 (job hunting, before the thumb-drive phase)						
`n_http_events`	173	162	162	**+11.0**	+1.8	−0.04
`n_email_sent`	12	6	8	+3.2	+4.4	−0.05
`n_distinct_domains`	27	23	26	+1.2	+0.3	−0.00
CDE1846, 25 April 2011 (another user's PC, files emailed to a personal address)						
`total_email_size`	46.4 MB	1.3 MB	4.8 MB	**+19.4**	+21.5	−0.03
`n_email_attachments`	48	1	6	+16.7	+20.0	−0.03
`n_email_external`	18	1	3	+15.2	+11.7	−0.07
`n_email_sent`	23	3	5	+13.8	+9.5	−0.04
`n_file_events`	8	0	1	**+12.2**	+1.4	−0.07

The answer to SQ5 is that a per-user deviation tells the analyst what changed, by how much relative to this person's own normal, and what that normal is. Measured against ground truth, that reason was right on 95% of malicious days and on every alert the framework raised. The Random Forest's SHAP explanations were right for most of its own alerts when read for positive contributions, but it raised far fewer alerts, and across the malicious

days as a whole they were no better than chance and carry no notion of the user's normal. Whether this shortens investigations in practice needs the user study proposed as future work.

6.9 The Dashboard

The research proposal promised a prototype that a non-technical security manager could use, and a prototype is a first working version. The dashboard is that product, and a reader can try it. The Streamlit dashboard has six pages. *Risk Explorer* plots a user's daily display score under both the framework and the Random Forest, where the display score runs from 0 to 1 and each model's alert line is drawn at the 0.1% budget. Every flagged day is explained against the user's own typical and 90th-percentile day (Figure 9a), while for the Random Forest the page shows the SHAP contributions that push towards an alert. *Method Comparison* shows who each model spends its alert budget on, role by role, and sets this against each role's share of user-days (Figure 9b). *Evaluation* scores the whole test window at a chosen budget, *Findings* presents the saved results of this report, and *Overview* and *About* complete the set. The dashboard's two models are trained on the full population with the settings of Table 3 and a contamination of 0.03. The dashboard runs as a packaged copy of the software with the trained models attached read-only, and it is served over HTTPS from Amazon Web Services (AWS) at the address on the title page.

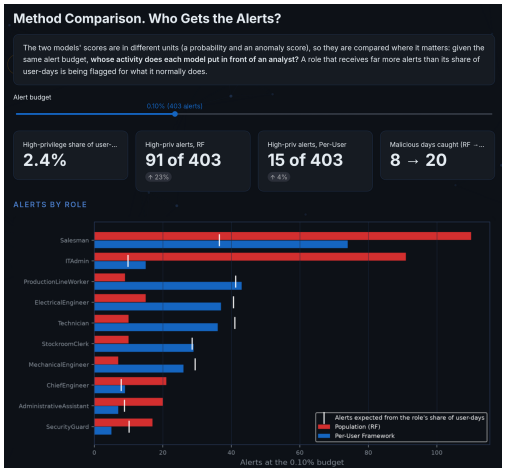
6.10 Unexpected and Negative Results

A no-model detector beats most comparison detectors. Ranking user-days by total activity uses no model at all, but it matches Logistic Regression's average precision of 0.0284 and beats the Random Forest's 0.0077. CMP2946's thumb-drive days in scenario 2 are simply busy days.

Two supervised comparison detectors fail outright. The Decision Tree is grown until every training row is classified, and on the test window it gives exactly two distinct scores, where two rows get probability 1 and the other 403 243 get 0. It therefore has no ranking, and 99.5% of its budget is a coin toss. Gradient Boosting reaches a ROC-AUC of 0.379, which is worse than random. The likely cause is the training window, because six burst examples are tested on low-and-slow behaviour.



(a) Risk Explorer for CMP2946. The framework, in blue, is above its alert line on 18 of the 29 malicious days. Each alert names what changed against the user's own normal.



(b) Method Comparison. IT administrators are 2.43% of user-days. Of 403 alerts, the Random Forest spends 91 on them and the framework 15.

Figure 9: The deployed dashboard, full population, 0.1% budget.

Two measures mislead in opposite ways. `pop-iforest` has a ROC-AUC of 0.935, the second highest in Table 6, but it finds no malicious day and spends half its queue on administrators, which means it orders the benign mass well and the top badly. The role-scoped framework shows the opposite, where it has the highest average precision, 0.0433, but finds 3 days to the personal framework's 21, because average precision is decided mostly by the very top of the ranking. No single number suffices, which is why recall and insider-level recall at the budget, the budget sweep and queue depth are all reported.

Many days share the top score. Far from every training point the model kept, the RBF kernel sum is zero, so the One-Class SVM's score equals its constant ρ, its highest possible value. In the test window, counted from the seed-42 scores, 140 user-days receive exactly this maximum score, and a further seven lie within 10^{-9} of it. Three of these 147 days are malicious, and one of them is in the exactly tied group. At the 0.1% budget all 147 are included and the effect is invisible, but at 40 alerts the selection is a seeded coin toss among the 140 tied rows. The tie fraction is 1.0, since every one of the 40 alerts is chosen by the tie-break, and that toss happened to draw no

malicious day. Ordering this top group by the sum of the absolute deviations in the vector would fix this, but it is left as future work.

6.11 Limits of the Evidence

Only two insiders. The test window contains 38 malicious days, and they belong to only two people. The seed study controls for chance inside the algorithms, but it cannot control for which insiders the dataset happens to contain. Insider-level claims rest on $n = 2$, and day-level recall moves in steps of 2.6 points. The results are evidence about these two low-and-slow scenarios, and they are not an estimate of performance on insiders in general.

How the algorithm was chosen. As Section 3.1 explains, the supervised comparison detectors learn from six examples of a different kind of attack, which puts them at a disadvantage. The population *unsupervised* controls see no labels either, so they are the fairer comparison, and the framework beats those too. A more serious limit is how the anomaly algorithm was chosen. It was chosen on the 400-user sample, and that sample's test window contains the same two insiders as the full test window. The framework was first built with an Isolation Forest, so the choice of One-Class SVM over it is informed by test-window behaviour, and the main recall is probably too high. With the Isolation Forest scorer, the personal reference frame still cuts the administrator alert share from 49.4% to 7.7%, and it finds 3 malicious days and one insider where the population control finds none, but it comes nowhere near the One-Class SVM's 21 days.

What the measures leave out. Labels are day-level, so a flagged day counts as a hit even if the analyst would still need to find the malicious events within it. The alert budget is applied over the whole test window after the fact, while a system in use must choose each day's cut-off without seeing the future. The high-privilege subgroup is defined by role, and in r6.2 it is exactly one role. The only high-privilege insider is PLJ1771 in scenario 3, and that insider acts in the training window, so this evaluation cannot show whether per-user scoring would also suppress a *malicious* administrator.

Generated data. CERT r6.2 is generated by a program from models of one made-up organisation [10]. At least one user, CMP2946, is perfectly regular on one feature and made 162 web requests on every training day. Such sameness flatters personal baselines, because small departures stand

out. Real logs are expected to be noisier, but this study did not test that, so results on real logs, and on other CERT releases, may be weaker.

6.12 What Changed From the Research Proposal

The research proposal set out the problem, the five sub-questions, the three-part framework, a dataset, a comparison detector, a set of measures, a schedule and a risk register. The problem statement and the sub-questions were carried into this report with only light rewording, while Tables 14 and 15 list every design choice that did not work.

Table 14: Defects that produced numbers of the right shape and the wrong meaning, each now locked in by a test.

Choice	Why it failed	Replacement	Section
Loading the raw logs into memory	93 GB of logs against 32 GB of memory	Read the logs once, in pieces	3.1
Reading every release's answer key	239 users labelled as insiders in a release with five	Filter to r6.2	3.1
Blaming every user named in the answer key	The 24 recipients of the scenario 3 email were labelled attackers	Blame only the named insider	3.1
Window-filled labels	95 user-days labelled where 44 are malicious	Exact day labels	3.1
Four feature definitions	Constant or broken on real data	Corrected and locked in by tests	3.1
Rescaled and capped risk score	Every row above the training range tied at 1.0, the whole top of the ranking	Model's own score for ranking, 0 to 1 score for display	3.2
Flagging every row tied at the cut	Random Forest flagged all 40 360 test rows of the 400-user sample against a 1% budget of 404	Seeded tie-break and tie fraction	3.4
First tie-fraction formula	Reported a non-zero tie share even when no rows were tied	Counting error of one row fixed	3.4
SMOTE seeded with 42 always	Seed study would not have re-drawn it	Takes the run's seed	3.7

Table 15: Modelling choices the controlled comparison rejected.

Choice	Why it failed	Replacement	Section
Median statistic	AP 0.030 against 0.292 for the z-score on the sample (suspected cause, zero IQRs raised to one)	z-score	6.4
Size channel	Administrator alert share 7.7% to 36.0%	Not used	6.3
Rolling-window features with the One-Class SVM	Recall 0.553 to 0.053	Base features	6.4
LOF as scorer	No malicious day found in any setting, slowest model	One-Class SVM	6.4
Isolation Forest as default scorer	3 malicious days against 21	One-Class SVM	6.4
14-day cold-start threshold	Recall 0.018 at 14 days against 0.342 at 60	About 60 days recommended	6.6
SHAP ranked by size ignoring sign	Top reason correct on 2.6% of malicious days, below chance	Positive contributions only	6.8

The data and the method. The research proposal described r6.2 as 1 000 users over 18 months with logon, file, device, print and email logs and psychometric scores, but the release has 4 000 users over 516 days, no print log, and a web log the research proposal did not mention. The Anomaly Scorer scores a user-day rather than "a new log record", so every feature is a daily count, and a per-user-day table is what makes a 93 GB log fit a laptop. The "configurable historical window" became the full training history, and its length was studied as SQ3, as the risk register said to do. The median statistic, the size channel and the rolling-window features, which the risk register kept in reserve, were all tried and rejected (Table 15). Beyond the research proposal's Random Forest, three more classifiers, three population anomaly controls and two no-model detectors were added, so that every difference could be traced to its cause. All the research proposal's measures are reported, at an alert budget rather than a fixed cut-off, because the detectors' scores are not on one scale. The research proposal's hypothesis ended "without significant loss in overall accuracy", but accuracy means nothing here, since a detector that flags nothing scores 99.99% when only 38 of the 403 245 test rows are malicious. The hypothesis in this report asks for a cost a laptop can carry instead. Both failures the research proposal predicted appeared in the data

(Sections 6.2 and 6.3), while a third it did not foresee, the over-flagging of executives, appeared too, and the same reference frame removed it.

The research proposal's sources. Each source was checked while writing this report, and six errors were found, which are listed so that no one relies on the research proposal's versions. The paper cited as Noever's "Machine learning classifiers and the insider threat" (arXiv:1908.05895) is in fact "Classifier Suites for Insider Threat Detection" (arXiv:1901.10948) [4]. The paper cited as Yuan et al., IEEE Transactions on Information Forensics and Security 2021, for a 31 per cent false positive reduction from role normalisation, is a 2018 conference paper on an LSTM and CNN classifier [27] that makes no such claim. The paper cited as Le and Zincir-Heywood 2020, for a user study in which SHAP raised analyst agreement by 23 per cent, is a 2018 workflow paper [11] with no SHAP user study. The paper cited as Liu, Xu, Shu and Liu (2019) in Computers and Security, for a 14 percentage point recall gain for per-user baselines, could not be found. The IBM 2024 report was given as the source of a 197-day detection time, but it gives 287 days [1]. The forensic readiness paper cited as a 2025 journal article by Shoderu, Zuva and Fadahunsi is a 2026 conference paper by Shoderu, Baror, Modupe and Makura [28]. This report uses none of those numbers. Two industry sources for alert fatigue and UEBA were replaced by a peer-reviewed interview study [5] and the vendor's own documentation [6]. The autoencoder the research proposal reviewed [29], a neural network that learns to copy its input, was not part of the planned comparison and was not built.

Schedule and hardware. The research proposal planned the work from February to October 2026, where the framework was to be built in May and June, the evaluation in July, and the prototype and report in August and September. The code history shows a first prototype on 24 August 2026, the data pipeline and first tests on 7 and 8 September, the follow-up studies and a first full report on 13 October, and deployment and documentation between 21 and 24 October. Those three phases were therefore delivered late and compressed into about seven weeks. The risk register's answer to delay was to drop one of the three algorithms, but that was not needed.

7 Conclusion

This research asked whether a lightweight per-user behavioural baseline could detect insider activity that a population-level detector misses. It also asked whether the baseline could do so while spending fewer alerts on users who are busy for good reason, and on hardware a resource-constrained organisation already owns. The baseline was evaluated on the full CERT r6.2 dataset, and it was compared against the research proposal's Random Forest, against the same anomaly algorithms with a population reference, and against no-model detectors.

Answers to the sub-questions.

- **SQ1, low-and-slow detection.** At a 0.1% alert budget the per-user baseline found 21 of 38 low-and-slow malicious days and both insiders. The Random Forest found 8 days, between 3 and 8 across ten seeds, and one insider, while the same algorithm with a population reference found 3 days. The gain is concentrated on the insider whose behaviour was unusual for themselves but ordinary for the organisation.
- **SQ2, high-privilege false positives.** IT administrators are 2.43% of user-days. The Random Forest, the population Isolation Forest, the population One-Class SVM and gradient boosting spent 21 to 49% of their alerts on them, while the framework spent 3.5%. Holding the algorithm fixed shows that the reference frame removes the bias, not the model, and the same holds for executives.
- **SQ3, history needed.** A personal baseline is stable after about 30 active days and useful after about 60, so the cold-start threshold should be about 60 days, not the 14-day default. The measurements suggest that 14 active days, about three weeks of work, is not yet a person's normal. Until 60 days are available, the role baseline is the fairer and the more effective reference.
- **SQ4, cost.** The framework trains on 990 765 user-days in about eight seconds on a laptop, which is 39 times faster than the SMOTE-balanced Random Forest. The one-off reduction of 93 GB of logs takes 3.3 minutes with 12 workers, or 15 minutes in under 3 GB of memory.
- **SQ5, analyst context.** A per-user deviation tells the analyst what changed, by how much relative to this person's normal, and what that normal is. Its top-named reason was correct on 94.7% of malicious test days, while a population reference managed 52.6%, and the Random Forest's SHAP values at most 50.0%. Both insiders sit within the

framework's first 200 alerts. Whether this shortens real investigations needs a user study.

Contributions. The work contributes a working framework and data pipeline that others can rerun, and a controlled evaluation design that other work on this dataset can reuse. It also contributes three follow-up studies, and a record of the negative results and of every change from the research proposal.

Limitations and future work. The evidence comes from one synthetic dataset and two test-window insiders, and the anomaly algorithm was selected on data containing those insiders. The only malicious administrator falls outside the test window, and the alert budget is set after the fact (Section 6.11). Five directions follow directly from the findings. The first is a daily alert budget that chooses each day's alerts from that day's scores, as a system in use must. The second is ordering the One-Class SVM's top-scored group by the size of the deviation vector. The third is baselines that switch from role to personal reference at the roughly 60 days SQ3 identifies. The fourth is selecting the algorithm on one CERT release and evaluating on others such as r4.2 and r5.2, or on real logs collected with permission. The fifth is a user study of whether the per-user explanations reduce investigation time.

Conclusion. Judging each employee against their own normal, rather than the organisation's, found a slow insider that the same algorithm with a population reference ranked thousands of alerts deep, and that insider was CMP2946. The same approach kept administrators and executives near their share of the alert queue, and it ran in seconds on a laptop. This is not a replacement for a security operations centre, but it is evidence that the core idea of commercial behaviour analytics is within reach of organisations that cannot buy one.

References

[1] IBM Security, "Cost of a data breach report 2024," tech. rep., IBM Corporation, 2024. [Online]. Available: https://www.ibm.com/reports/data-breach.

[2] D. M. Cappelli, A. P. Moore, and R. F. Trzeciak, *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes*. Addison-Wesley Professional, 2012. ISBN 978-0-321-81257-5. [Online]. Available: https://www.oreilly.com/library/view/the-cert-guide/9780132906050/.

[3] I. Homoliak, F. Toffalini, J. Guarnizo, Y. Elovici, and M. Ochoa, "Insight into insiders and IT: A survey of insider threat taxonomies, analysis, modeling, and countermeasures," *ACM Computing Surveys*, vol. 52, no. 2, pp. 30:1–30:40, 2019. doi:10.1145/3303771.

[4] D. Noever, "Classifier suites for insider threat detection." arXiv preprint arXiv:1901.10948, 2019. [Online]. Available: https://arxiv.org/abs/1901.10948.

[5] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *31st USENIX Security Symposium*, pp. 2783–2800, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi.

[6] Microsoft, "Advanced threat detection with user and entity behavior analytics (UEBA) in Microsoft Sentinel." Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/sentinel/identify-threats-with-entity-behavior-analytics (accessed October 2026).

[7] A. Tuor, S. Kaplan, B. Hutchinson, N. Nichols, and S. Robinson, "Deep learning for unsupervised insider threat detection in structured cybersecurity data streams," in *AAAI Workshop on Artificial Intelligence for Cyber Security*, pp. 224–231, 2017. [Online]. Available: https://arxiv.org/abs/1710.00811.

[8] D. C. Le, N. Zincir-Heywood, and M. I. Heywood, "Analyzing data granularity levels for insider threat detection using machine learning," *IEEE Transactions on Network and Service Management*, vol. 17, no. 1, pp. 30–44, 2020. doi:10.1109/TNSM.2020.2967721.

[9] B. Lindauer, "Insider threat test dataset." Carnegie Mellon University, KiltHub. doi:10.1184/R1/12841247.v1, 2020.

[10] J. Glasser and B. Lindauer, "Bridging the gap: A pragmatic approach to generating insider threat data," in *IEEE Security and Privacy Workshops (SPW)*, pp. 98–104, 2013. doi:10.1109/SPW.2013.37.

[11] D. C. Le and A. N. Zincir-Heywood, "Evaluating insider threat detection workflow using supervised and unsupervised learning," in *IEEE Security and Privacy Workshops (SPW)*, pp. 270–275, 2018. [Online]. Available: `https://ieeexplore.ieee.org/document/8424659`.

[12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: `https://jmlr.org/papers/v12/pedregosa11a.html`.

[13] S. Yuan and X. Wu, "Deep learning for insider threat detection: Review, challenges and opportunities," *Computers & Security*, vol. 104, p. 102221, 2021. doi:10.1016/j.cose.2021.102221.

[14] P. J. Rousseeuw and M. Hubert, "Robust statistics for outlier detection," *WIREs Data Mining and Knowledge Discovery*, vol. 1, no. 1, pp. 73–79, 2011. doi:10.1002/widm.2.

[15] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009. doi:10.1145/1541880.1541882.

[16] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pp. 413–422, 2008. doi:10.1109/ICDM.2008.17.

[17] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 93–104, 2000. doi:10.1145/342009.335388.

[18] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001. doi:10.1162/089976601750264965.

[19] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. doi:10.1613/jair.953.

[20] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 233–240, 2006. doi:10.1145/1143844.1143874.

[21] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015. doi:10.1371/journal.pone.0118432.

[22] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 4765–4774, 2017. [Online]. Available: `https://arxiv.org/abs/1705.07874`.

[23] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020. doi:10.1038/s42256-019-0138-9.

[24] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945. doi:10.2307/3001968.

[25] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006. [Online]. Available: `https://jmlr.org/papers/v7/demsar06a.html`.

[26] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: `https://www.jstor.org/stable/4615733`.

[27] F. Yuan, Y. Cao, Y. Shang, Y. Liu, J. Tan, and B. Fang, "Insider threat detection with deep neural network," in *Computational Science – ICCS 2018*, vol. 10860 of *Lecture Notes in Computer Science*, Springer, 2018. doi:10.1007/978-3-319-93698-7_4.

[28] G. Shoderu, S. Baror, A. Modupe, and S. Makura, "Digital forensic readiness to mitigate insider threats in the SaaS cloud environment," in *Proceedings of the 21st International Conference on Cyber Warfare and Security (ICCWS)*, vol. 21, pp. 427–438, 2026. doi:10.34190/iccws.21.1.4508.

[29] S. Hawkins, H. He, G. Williams, and R. Baxter, "Outlier detection using replicator neural networks," in *Data Warehousing and Knowledge Discovery (DaWaK)*, vol. 2454 of *Lecture Notes in Computer Science*, pp. 170–180, 2002. doi:10.1007/3-540-46145-0_17.